Développer en back-end

les relations

Réalisé par M. Hamid Belyaz

Développer en back-end

Introduction

A. Découvrir le Framework PHP Laravel

1. Découvrir les notions fondamentales des Frameworks PHP
2. Préparer l'environnement de Laravel

B. Programmer avec Laravel

1. Connaître les fondements du modèle MVC Laravel
2. Maîtriser le Framework Laravel

C. Approfondir la programmation Laravel

1. Gérer la sécurité
2. Interagir avec la base de données
3. Manipuler l'ORM Eloquent
4. Prendre en charge les tests

D. Administrer un site à l'aide d'un CMS

1. Manipuler les éléments essentiels d'un CMS
2. Personnaliser graphiquement un site à l'aide d'un CMS
3. Manipuler les outils avancés d'un CMS

Conclusion

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

- ✓Intérêt de l'Object Relational Mapper ORM Eloquent
- ✓Manipulation des modèles (Génération, récupération, insertion, suppression…)
- ✓**Relations Eloquent**
- ✓Manipulation des collections
- ✓Transformation des valeurs d'attributs Eloquent (accessor, mutator, casting)
- ✓Sérialisation

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Definition

Une relation signifie que vous avez deux tables ou plus avec des enregistrements qui sont liés les uns aux autres .

Les relations Eloquent sont définies comme des méthodes sur vos classes de modèle Eloquent.

Étant donné que les relations servent également de puissants générateurs de requêtes , la définition de relations en tant que méthodes offre de puissantes capacités de chaînage de méthodes et d'interrogation.

Développer en back-end

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Types de relations dans ORM Eloquent

Partie I

- Relation (***One To One) 1 à 1***
- Relation (***One To Many) 1 à plusieurs***
- Relation (***Many To Many) plusieurs à plusieurs (avec table intermédiaire)***

Partie II

- Relation ***(Has One Through)***
- Relation (***Has Many Through)***

Partie III

- Relation (***One To One Polymorphic)***
- Relation (***One To Many Polymorphic)***
- Relation (***Many To Many Polymorphic)***

C. Approfondir la programmation Laravel

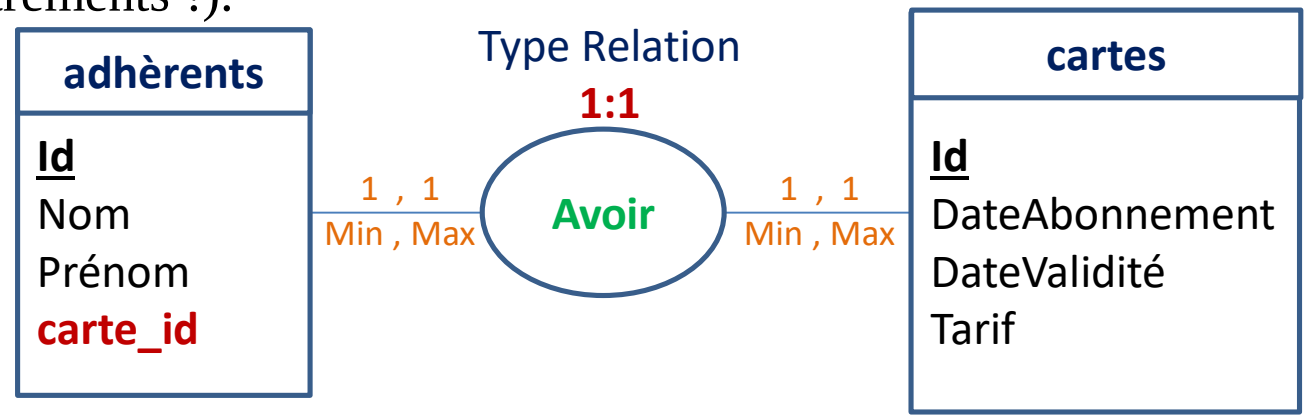
3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

La plus simple relation mais la moins utile. Une relation de type « **Has One** » ou « **One-To-One** ». Cela signifie effectivement qu'un certain enregistrement est lié à un autre enregistrement (mais pas à plusieurs autres enregistrements !).

Exemple :



C. Approfondir la programmation Laravel

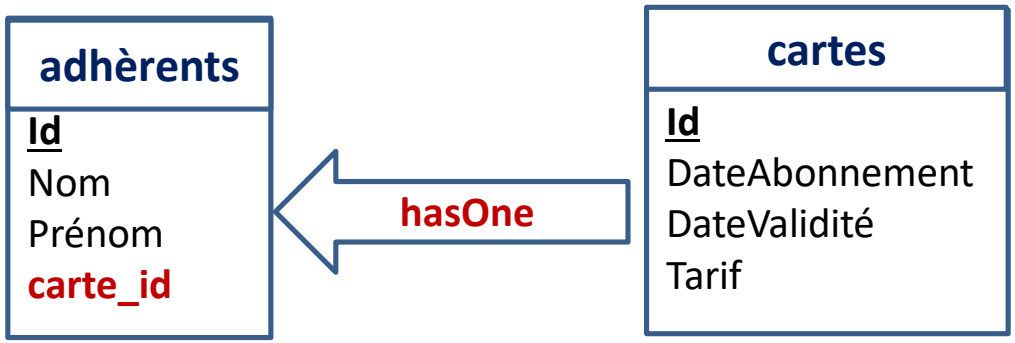
3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

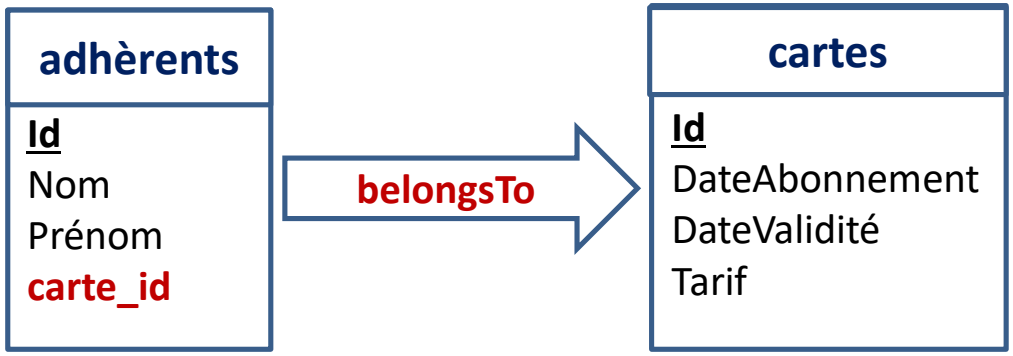
- ✓ la relation directe :

hasOne



- ✓ l'inverse de la relation :

belongsTo



C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

Migration : cartes

```
public function up()
 {
    Schema::create('cartes', function (Blueprint $table) {
            $table->id();
            $table->date('DateAbonnement');
            $table->date('DateValidité');
            $table->double('Tarif');
            $table->timestamps();
        });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

Migration : adherents

```
public function up()
 {
  Schema::create('adherents', function (Blueprint $table){
    $table->id();
    $table->string('Nom');
    $table->string('Prenom');
    $table->foreignId('carte_id')->constrained()->cascadeOnDelete;
  //$table->foreignId('carte_id')->references('id')->on('cartes');
    $table->timestamps();
 });
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

Model : Carte

```php
class Carte extends Model
{
    use HasFactory;
    public function adherent(){
        return $this->hasOne(Adherent::class);
    }
}
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

Model : Adherent

```php
class Adherent extends Model
{
    use HasFactory;
    public function carte(){
        return $this->belongsTo(Carte::class);
    }
}
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To One)

Contrôleur: CarteController

```php
public function index()
    {
        //$carte = Carte::find(2);
        $adherent = Adherent::find(1);
        return $adherent->carte;
        //return $carte->adherent;
    }
```

routes : web.php

```php
Route::resource('cartes', CarteController::class);
```

C. Approfondir la programmation Laravel

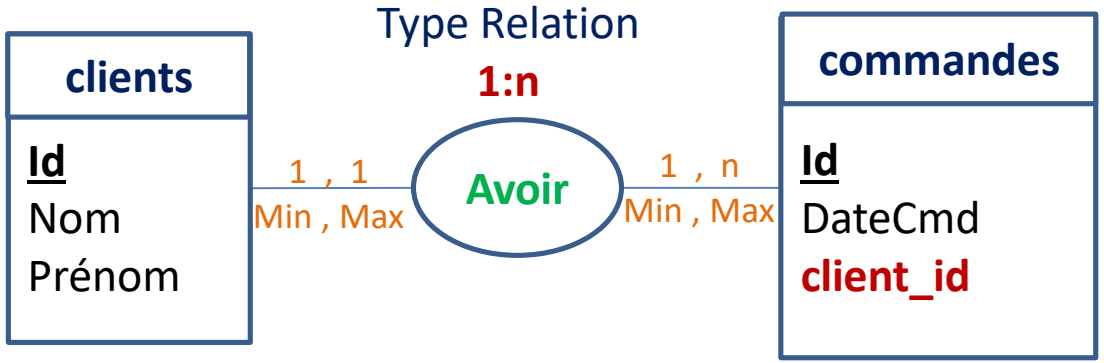
3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

Il s'agit d'une relation très importante, peut-être même la plus importante. Une relation telle que « **Has Many** » ou « **One-To-Many** ». Cela signifie en fait qu'un certain enregistrement est lié à plusieurs autres enregistrements.

Exemple :



On a la situation suivante :

- ✓ un client peut avoir plusieurs commandes .
- ✓ une commande ne concerne qu'un seul client.

C. Approfondir la programmation Laravel

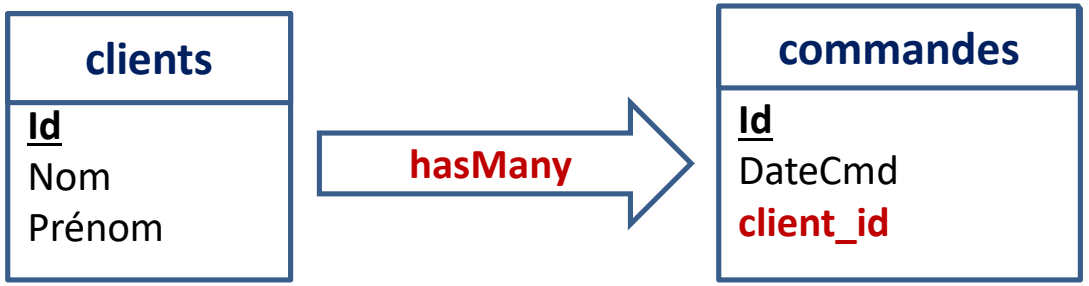
3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

- ✓ la relation directe:

hasMany



- ✓ l'inverse de la relation :

belongsTo



C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

Migration : clients

```
public function up()
 {
    Schema::create('clients', function (Blueprint $table)
        {
            $table->id();
            $table->string('nom');
            $table->string('prenom');
            $table->timestamps();
         });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

Migration : commandes

```
public function up()
{
   Schema::create('commandes', function (Blueprint $table)
        {
            $table->id();
            $table->date('DateCmd');
            $table->foreignId('client_id')->
               constrained('clients')->cascadeOnDelete();
            $table->timestamps();
        });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

Model : Client

```php
class Client extends Model
{
    use HasFactory;
    public function commandes(){
        return $this->hasMany(Commande::class);
    }
}
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (One To Many)

Model : Commande

```
class Commande extends Model
{
    use HasFactory;
    public function client(){
        return $this->belongsTo(Client::class);
    }
}
```

Développer en back-end

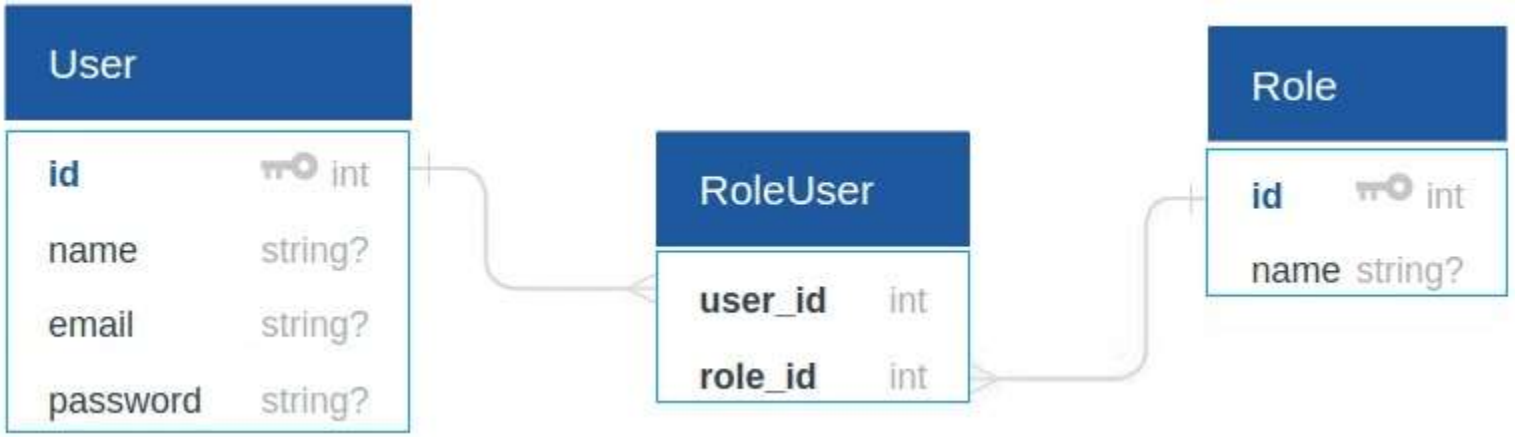
C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Les relations **n..n (Many To Many)** permettent de définir une relation entre plusieurs objets d'un côté et plusieurs objets de l'autre.



On a la situation suivante :

- ✓ Un utilisateur peut avoir plusieurs rôles ('admin' et 'modérateur').
- ✓ Un rôle peut être donné à plusieurs utilisateurs('yahya' , 'ritaje').

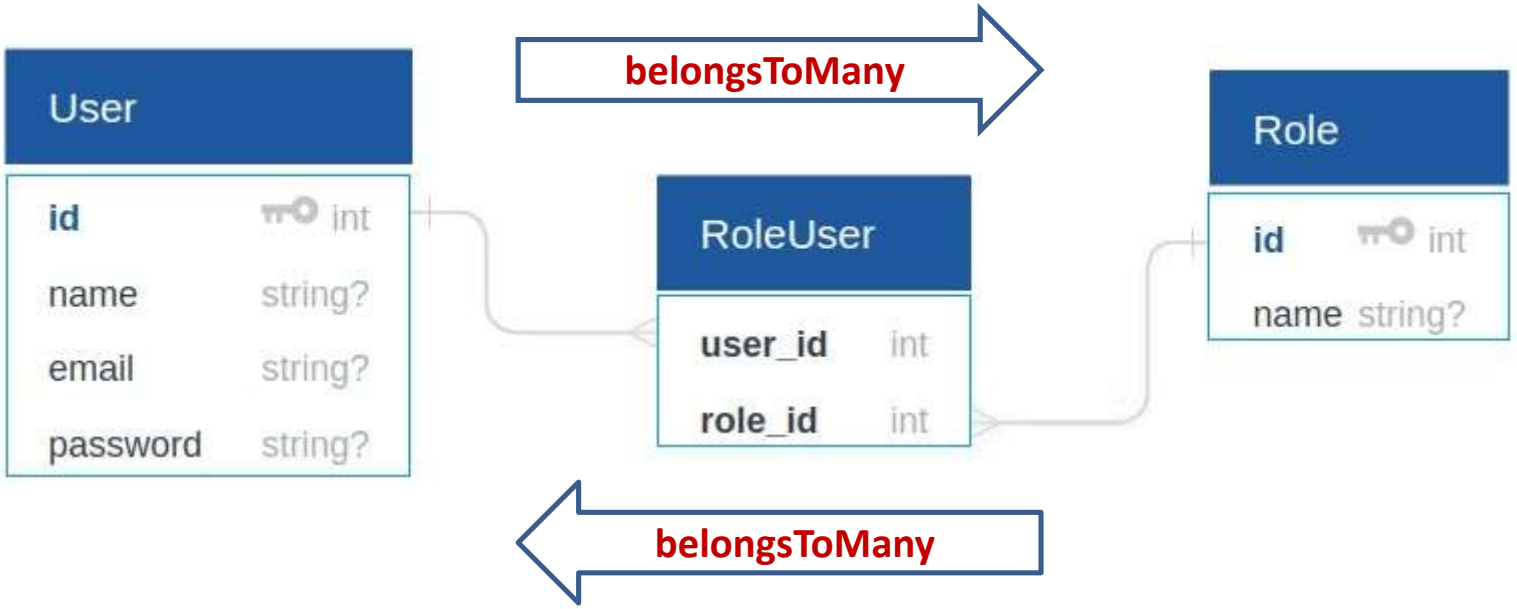
C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

- ✓ Comme la relation est symétrique on a une seule méthode : belongsToMany



C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Création des modèles

```
//Création des modèles
php artisan make:model User --migration
php artisan make:model Role --migration
```

Il faut donc au préalable créer également une migration pour la table **role_user**. le nom de la table est construit en joignant les noms des tables ***(tous deux au singulier !)*** dans l'ordre alphabétique. Cela permet à Laravel de faire directement le lien entre les tables sans avoir à préciser le nom de la table de relation.

```
//Création de la table de pivot
php artisan make:migration create_role_user_table
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Migration : users

```
public function up()
 {
    Schema::create('users', function (Blueprint $table)
        {
             $table->id();
             $table->string('name');
             $table->string('email');
              $table->string('password');
             $table->timestamps();
         });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Migration : roles

```
public function up()
 {
    Schema::create('roles', function (Blueprint $table)
        {
            $table->id();
            $table->string('name');
            $table->timestamps();
         });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Migration : la table pivot : role_user

```
public function up()
{
  Schema::create('role_user', function (Blueprint $table)
        {
          $table->id();
          $table->timestamps();

          $table->foreignId('role_id')->constrained();
          $table->foreignId('user_id')->constrained();
        });
    }
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Model : User

Les méthodes sur chaque modèle sont toutes deux de la forme belongsToMany().

```
class User extends Model
{
    use HasFactory;
    public function roles(){
        return $this->belongsToMany(Role::class);
 }
}
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Model : Role

Les méthodes sur chaque modèle sont toutes deux de la forme belongsToMany().

```
class Role extends Model
{
    use HasFactory;
    public function users(){
        return $this->belongsToMany(User::class);
 }
}
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Si vous choisissez une autre table que le nom par défaut, vous pouvez ajouter votre propre nom de table en tant que paramètre à la méthode d'extraction.

```
return $this->belongsToMany(User::class, 'user_role');
```

Par défaut, Eloquent suppose que le **user_id** et **role_id** colonnes de la table croisé dynamique. Si vous choisissez de les nommer différemment, vous pouvez l'utiliser comme troisième et quatrième paramètre :

```
return $this->belongsToMany(Profile::class, 'user_role',
'user_id', 'role_id');
```

C. Approfondir la programmation Laravel

3. Manipuler l'ORM Eloquent

les relations de modèle dans Laravel Eloquent

Relation (Many To Many)

Contrôleur: UserController

```
public function index()
    {
        //$user = User::find(1);
       //return $user->roles;
        $role = Role::find(1);
        return $role->users;
    }
```

routes : web.php

```
Route::resource('users', UserController::class);
```